并发编程之synchronize原理-对象头

如何自己实现同步

https://blog.csdn.net/java_lyvee/article/details/98966684

利用park+自旋来实现同步

```
volatile int status=0;
Queue parkQueue;//集合 数组  list

void lock(){
    while(!compareAndSet(0,1)){
        //
        park();
    }
    //lock    10分钟
   。。。。。。
   unlock()
}

void unlock(){
    lock_notify();
}

void park(){
    //将当期线程加入到等待队列
    parkQueue.add(currentThread);
    //将当期线程释放cpu  阻塞
    releaseCpu();
}
void lock_notify(){
    //得到要唤醒的线程头部线程
    Thread t=parkQueue.header();
    //唤醒等待线程
    unpark(t);
}
```

如果你能模拟一把锁，你就大概知道了锁其实很简单，就是一个对象里面用了一个变量来标识是否自由状态；所谓加锁就是改变这个对象当中的这个变量；status=1；所谓解锁就是把这个变量的值改为0；如果加锁成功就正常返回；如果加锁不成功就死循环或者阻塞；

关于自己实现一把最简单的锁可以参考我发的源码当中的注释

java头的信息分析

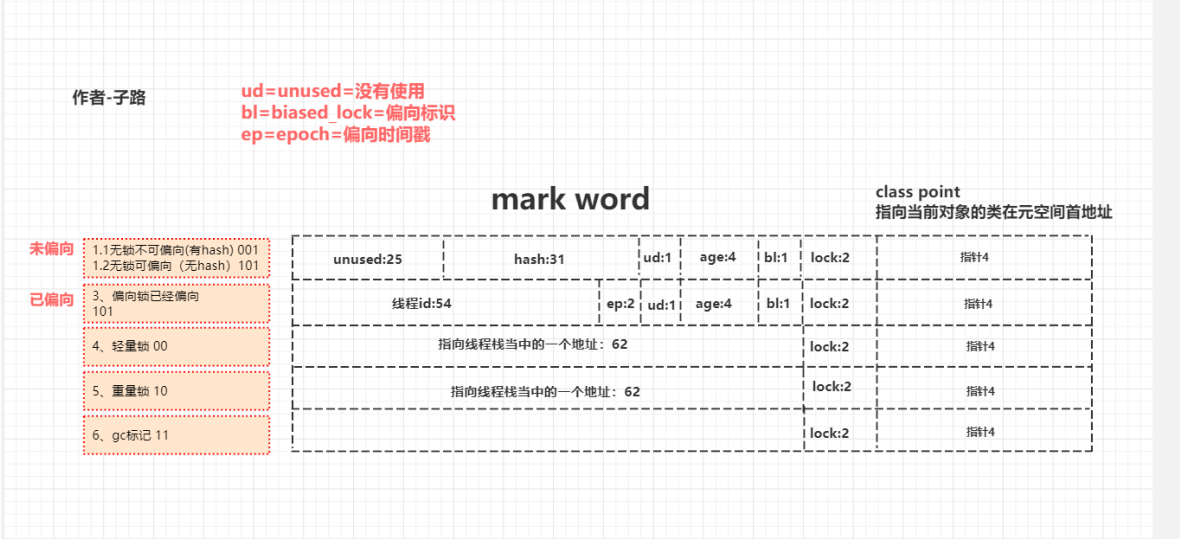
我们已经自己实现了一把锁；那么关于java当中的内置锁synchronized是怎么实现的？
首先为什么我要去研究java的对象头

因为所谓锁便是给对象一个标识；而这个标识便是存在对象头当中

这里截取一张hotspot的源码当中的注释

```
//  64 bits:
//  --------
//  unused:25 hash:31 -->| unused:1   age:4    biased_lock:1 lock:2 (normal object)
//  JavaThread*:54 epoch:2 unused:1   age:4    biased_lock:1 lock:2 (biased object)
//  PromotedObject*:61 --------------------->| promo_bits:3 ----->| (CMS promoted object)
//  size:64 ----------------------------------------------------->| (CMS free block)
//
```

这张图换成人可读的表格如下



意思是java的对象头在对象的不同状态下会有不同的表现形式，主要有三种状态，无锁状态、加锁状态、gc标记状态。那么我可以理解java当中的取锁其实可以理解是给对象上锁，也就是改变对象头的状态，如果上锁成功则进入同步代码块。但是java当中的锁有分为很多种，从上图可以看出大体分为偏向锁、轻量锁、重量锁三种锁状态。这三种锁的效率完全不同，我们只有合理的设计代码，才能合理的利用锁、那么这三种锁的原理是什么?所以我们需要先研究这个对象头。

java对象的布局以及对象头

1、JOL来分析java的对象布局

首先maven添加JOL的依赖

```
<dependency>
    <groupId>org.openjdk.jol</groupId>
    <artifactId>jol-core</artifactId>5
    <version>0.9</version>
</dependency>
```

A.java 源码

```
package com.enjoy.entity;
public class A {
}
```

JOLExample1.java

```
package com.enjoy.test;

import com.enjoy.entity.A;
import org.openjdk.jol.info.ClassLayout;
import org.openjdk.jol.vm.VM;
```

```java
public class Test1 {
    static A a= new A();
    public static void main(String[] args) {
        System.out.println(VM.current().details());
        System.out.println(ClassLayout.parseClass(A.class).toPrintable());
        System.out.println(ClassLayout.parseInstance(a).toPrintable());
    }
}
```

运行结果：

```
# Field sizes by type: 4, 1, 1, 2, 2, 4, 4, 8, 8 [bytes]
# Array element sizes: 4, 1, 1, 2, 2, 4, 4, 8, 8 [bytes]

com.enjoy.entity.A object internals:
 OFFSET  SIZE   TYPE DESCRIPTION                               VALUE
      0    12        (object header)                           N/A
     12     4        (loss due to the next object alignment)
Instance size: 16 bytes
Space losses: 0 bytes internal + 4 bytes external = 4 bytes total

com.enjoy.entity.A object internals:
 OFFSET  SIZE   TYPE DESCRIPTION                               VALUE
      0     4        (object header)                           05 00 00 00 (00000101 00000000 00000000 00000000) (5)
      4     4        (object header)                           00 00 00 00 (00000000 00000000 00000000 00000000) (0)
      8     4        (object header)                           48 72 06 00 (01001000 01110010 00000110 00000000) (4224
     12     4        (loss due to the next object alignment)
```

分析结果1

整个对象一共16B，其中对象头（Object header）12B，还有4B是对齐的字节（因为在64位虚拟机上对象的大小必须是8的倍数）,由于这个对象里面没有任何字段，故而对象的实例数据为0B；两个问题
1、什么叫做对象的实例数据呢？
2、那么对象头里面的12B到底存的是什么呢？
首先要明白什么对象的实例数据很简单，我们可以在A当中添加一个boolean的字段，大家都知道boolean字段占1B，然后再看结果
A.java

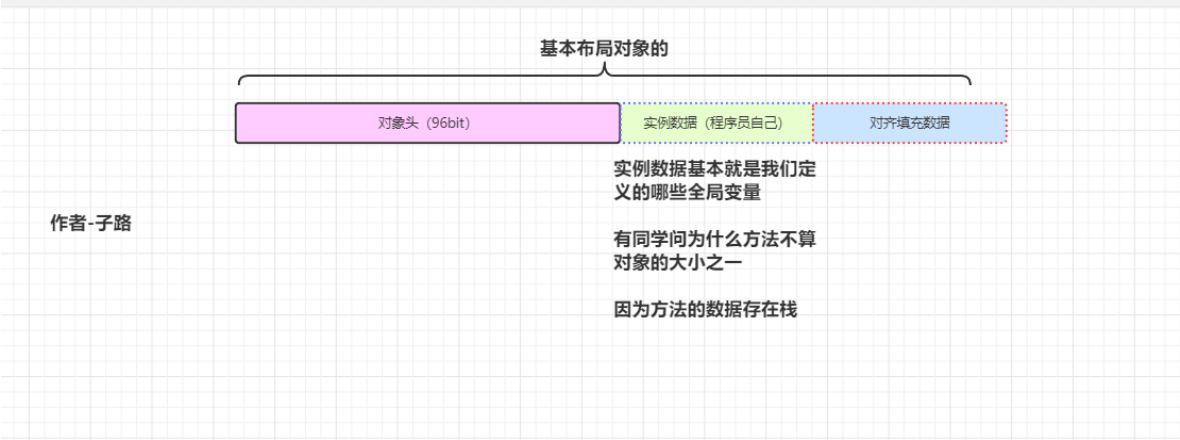
```java
package com.enjoy.entity;
public class A {
    boolean f;
}
```

运行结果2

```
14:05:50.805 [main] DEBUG enjoy - com.enjoy.entity.A object internals:
OFFSET  SIZE      TYPE DESCRIPTION                               VALUE
     0     4           (object header)                           05 00 00 00 (00000101 00000000 00000000 00000000) (5)
     4     4           (object header)                           00 00 00 00 (00000000 00000000 00000000 00000000) (0)
     8     4           (object header)                           99 67 01 f8 (10011001 01100111 00000001 11111000) (-134125671
    12     1   boolean A.f                                       false
    13     3           (loss due to the next object alignment)
Instance size: 16 bytes
Space losses: 0 bytes internal + 3 bytes external = 3 bytes total
```

分析结果2
整个对象的大小还是没有改变一共16B，其中对象头（Object header）12B，boolean字段f（对象的实例数据）占1B、剩下的3B就是对齐字节。由此我们可以认为一个对象的布局大体分为三个部分分别是对象头（Object header）、对象的实例数据字节对齐；

对象布局：

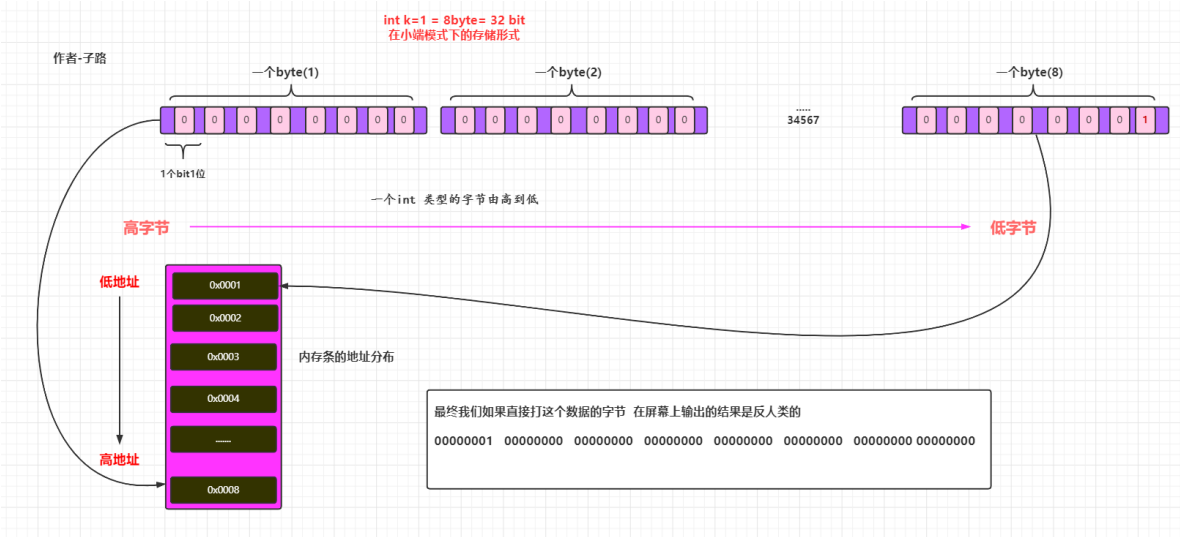


接下来讨论第二个问题，对象头为什么是12B？这个12B当中分别存储的是什么呢？（不同位数的VM对象头的长度不一样，这里指的是64bit的vm）

2、大小端模式

再接着往下研究对象的时候先要高明白大小端存储，一般家用笔记本都是小端模式，那么什么是小端模式呢？

高字节存在高地址，低字节存在低地址

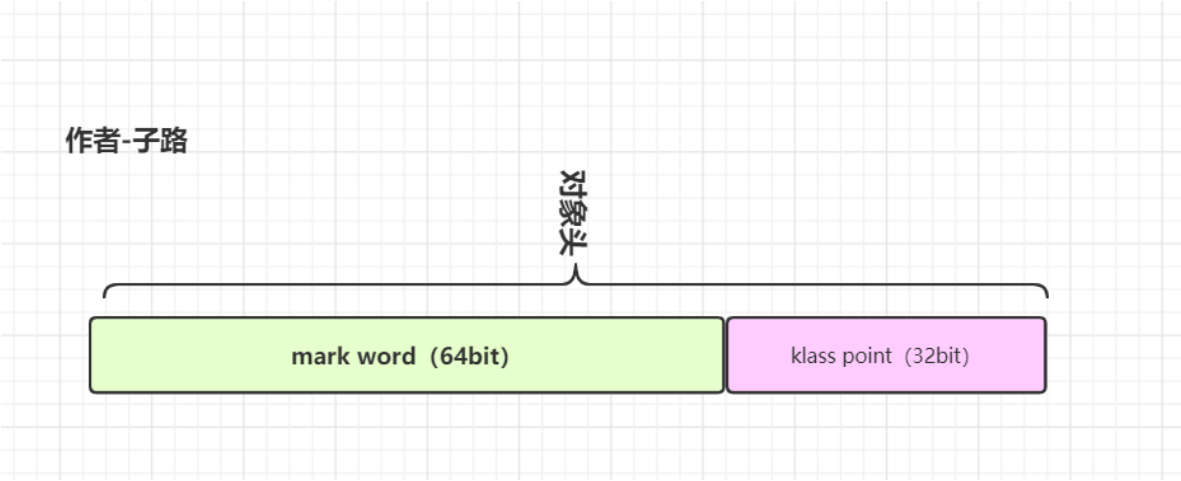


3、对象头的规范

http://openjdk.java.net/groups/hotspot/docs/HotSpotGlossary.html

首先引用openjdk文档当中对对象头的解释

上述引用中提到一个java对象头包含了2个word，并且好包含了堆对象的布局、类型、GC状态、同步状态和标识哈希码，具体怎么包含的呢？又是哪两个word呢？
对象头的结构大概如下图



mark word为第一个word根据文档可以知他里面包含了锁的信息，hashcode，gc信息等等，第二个word是什么呢?

klass word为对象头的第二个word主要指向对象的元数据，他是一个指针；

假设我们理解一个对象头主要上图两部分组成（数组对象除外，数组对象的对象头还包含一个数组长度），那么一个java的对象头多大呢？我们从JVM的源码注释中得知到一个mark word一个是64bit，那么klass的长度是多少呢？（12-8=4byte）

所以我们需要想办法来获得java对象头的详细信息，验证一下他的大小，验证一下里面包含的信息是否正确。
根据上述利用JOL打印的对象头信息可以知道一个对象头是12B，其中8B是mark word 那么剩下的4B就是klass word了，和锁相关的就是mark word了，那么接下来重点分析mark word里面信息

在无锁的情况下markword当中的前56bit存的是对象的hashcode，那么来验证一下
java代码

```
public class Test1 {
    static A a= new A();
    //-XX:BiasedLockingStartupDelay=0
    public static void main(String[] args) throws InterruptedException {
        log.debug("----hashcode before");
        log.debug(ClassLayout.parseInstance(a).toPrintable());
        //转化成16进制，方便比较
        log.debug(Integer.toHexString(a.hashCode()));
        log.debug("----hashcode after");
        //计算完hashcode之后的a对象的布局
        log.debug(ClassLayout.parseInstance(a).toPrintable());
    }
}
```

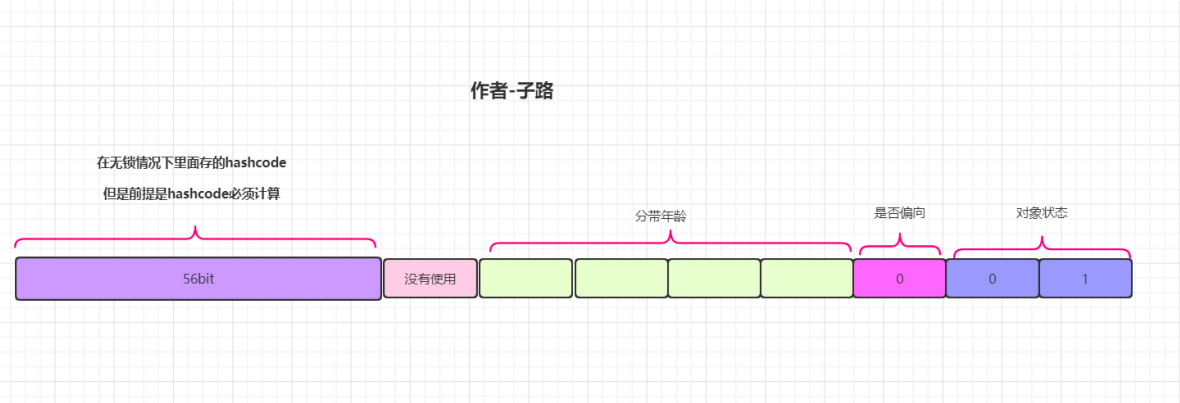
运行结果

```
"D:\Program Files\jdk11\jdk-11.0.5\bin\java.exe" "-javaagent:D:\Program Files\ideaIU-2019.2.4.win\lib\idea_rt.jar=5524
17:21:58.126 [main] DEBUG e - ----hashcode before
# WARNING: Unable to get Instrumentation. Dynamic Attach failed. You may add this JAR as -javaagent manually, or suppl
17:21:59.836 [main] DEBUG e - com.enjoy.entity.A object internals:
 OFFSET  SIZE   TYPE DESCRIPTION                               VALUE
      0     4        (object header)                           05 00 00 00 (00000101 00000000 00000000 00000000) (5)
      4     4        (object header)                           00 00 00 00 (00000000 00000000 00000000 00000000) (0)
      8     4        (object header)                           f8 31 0b 00 (11111000 00110001 00001011 00000000) (7336
     12     4    int A.f                                       0
Instance size: 16 bytes
Space losses: 0 bytes internal + 0 bytes external = 0 bytes total

17:21:59.836 [main] DEBUG e - 482cd91f
17:21:59.836 [main] DEBUG e - ----hashcode after
17:21:59.837 [main] DEBUG e - com.enjoy.entity.A object internals:
 OFFSET  SIZE   TYPE DESCRIPTION                               VALUE
      0     4        (object header)                           01 1f d9 2c (00000001 00011111 11011001 00101100) (7524
      4     4        (object header)                           48 00 00 00 (01001000 00000000 00000000 00000000) (72)
      8     4        (object header)                           f8 31 0b 00 (11111000 00110001 00001011 00000000) (7336
```

http://tool.oschina.net/hexconvert/ 进制转换

分析结果3：
没有进行hashcode之前的对象头信息，可以看到2b-8b之前的的56bit是没有值，打印完hashcode之后行就有值了，为什么是2-8B，不应该是1-7B呢？因为是小端存储。那么第一个字节当中的八位分别存的就是分带年龄、偏向锁信息，和对象状态，这个8bit分别表示的信息如下图;这个图会随着对象状态改变而改变，下图是无锁状态下



其中的是否可偏向标识在无锁情况下会根据是否计算hashcode而变化；因为如果计算了hashcode之后对象便变得不可偏向；为什么？

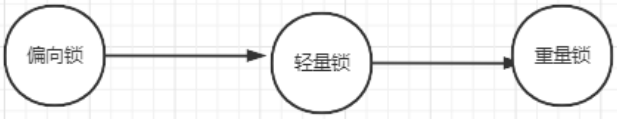
关于对象状态一共分为五种状态，分别是无锁不可偏向、无锁可向锁、轻量锁、重量锁、GC标记，那么2bit，如何能表示五种状态（2bit最多只能表示4中状态分别是：00,01,10,11），jvm做的比较好的是把是否可偏向表示为一个状态，然后根据图中偏向锁的标识再去标识是可偏向的还是不可偏向的

关于对象头目前只要了解这些就够了，下节课来解释什么是偏向锁，工作原理是什么和轻量锁的性能对比

这节课只要知道对象头里面存了什么东西就可以了；

关于课上做的那个锁的膨胀的实验大家可以自己用JOL去看看，锁的膨胀非常复杂，我只是先大概的给大家演示了一下膨胀过程

过程具体大概如下图（切记这个图不能作为面试，只是一个过渡结果，实际锁的膨胀远比这个复杂的多了多）



当一把锁第一次被线程持有的时候是偏向锁;如果这个线程再次加锁还是偏向锁
如果别的线程来加锁（交替执行）膨胀为轻量锁
如果是资源竞争膨胀重量锁
过程不可逆-----90%